```
  rescue BaseServiceWrapper::HTTPError => ex
    raise IgnorableExceptionSinceSidekiqWillRetry.new(ex)
  end
end
```

But what if someone uses `bin/rails g job` to make a new job?

Changing the Rails Job Generator to Use Sidekiq

Not every developer uses the Rails generator commands, but if you or anyone on your team does, you should make sure it generates a new job that uses Sidekiq and not Active Job. By default, Rails uses an internal template to create a job.[25] If your Rails app has the file `lib/templates/rails/job/job.rb.tt` in it, *that* template will be used. Despite the odd filename extention, this is an ERB file.

Since `--queue` allows specifying the queue when generating the job, your job should use that value with `sidekiq_options`. It's also good practice to have generated code raise an error. This way, if you forget to implement the job, it will fail loudly in testing or production. Note that `module_namespacing` handles a situation where you put your job in a sub-module.

snapshots/4-2/sidekiq-book/lib/templates/rails/job/job.rb.tt

```
<% module_namespacing do -%>
class <%= class_name %>Job < ApplicationJob
<% if options[:queue].to_s != "default" -%>
  sidekiq_options queue: "<%= options[:queue] %>"

<% end -%>
  def perform(args,should,go,here)
    raise "This job has not been implemented"
  end
end
<% end -%>
```

You should also override the test case that is generated. Even though you aren't likely to want to unit test your jobs, the default unit test includes Active